

UnifyComply API — frontend integration guide

Recorded 2026-09-14. The complete reference for building a frontend against core-platform, the only service clients ever call — compliance-domain, provider-gateway, risk-decisioning and transaction-monitoring all sit behind it and are never reached directly.

This is a route-and-behavior map, not a field-by-field schema dump. Every route's exact request/response types (including validators) are in Swagger, served live by core-platform at `/v1/docs`. Read this doc for what a route does, who can call it, and the gotchas that don't show up in a generated schema; read Swagger for the exact shape of a body you're about to send.

Companion docs: `kyc-review.md` / `kyb-review.md` (list/detail screen gap analysis), `verification-list-endpoints.md` (the 2026-09-14 verification-list change in detail), `risk-model.md` (score → band → priority), `geography.md` (country-agnostic verification types).

Contents

1. [Getting started](#) — base URL, versioning, envelope, pagination, errors
2. [Authentication](#) — two modes, how they differ, MFA
3. [Public option sets](#) — the enum-as-data endpoint
4. [Tenant onboarding](#)
5. [Apps, API keys & configuration](#)
6. [Domain \(sandbox / production\)](#)
7. [Customers — KYC & KYB](#)
8. [Verifications](#)
9. [Available checks](#)
10. [Dashboard](#)
11. [Webhooks](#)
12. [Transactions](#)
13. [User profile](#)
14. [Settings — deprecated tenant-wide routes](#)

1. Getting started

Base URL: `{API_BASE}/v1` — every route in this doc is shown without the `/v1` prefix; core-platform uses URI versioning (`enableVersioning`), so the literal path is `/v1/customers/kyc`, not `/customers/kyc`.

CORS: the server only accepts `Authorization`, `Content-Type` and `x-app-id` as request headers, and only `GET/POST/PUT/DELETE/PATCH/OPTIONS`. Your origin has to be allow-listed in core-platform's `CORS_ORIGINS`.

Request bodies: `application/json`. Documents are sent as base64 data URIs inline in JSON, not multipart — see [§7](#).

Response envelope, on every route, success or failure:

```
{
  "status": true,
  "message": "Customers kyc retrieved successfully",
  "data": /* the payload — object, array, or null */,
```

```
"meta": { /* present only on paginated list endpoints, see below */ }
}
```

Errors use the same envelope with `status: false` and an HTTP status code matching the failure (400 validation, 401/403 auth, 404 not found, 500 unexpected). `data` sometimes carries structured detail (e.g. which fields were invalid) — check it before falling back to just showing `message`.

Pagination. Any endpoint accepting `page / limit` returns:

```
"meta": {
  "totalItems": 42,
  "itemsCount": 20,    // rows actually returned on this page
  "itemsPerPage": 20,
  "totalPages": 3,
  "currentPage": 1
}
```

`limit` defaults to the server's configured page size if omitted (currently 20); there's no documented hard cap, but don't request more than a screen needs.

2. Authentication

Two independent modes, both accepted on any route guarded by `CombinedAuthGuard` (it tries a session JWT first, then falls back to an API key). A route guarded only by `PermissionsGuard` (no `CombinedAuthGuard`) is **session-only** — dashboard/back-office routes, never callable with an API key. See [§14](#) for which routes that applies to.

Session auth (the dashboard/back-office)

A human signs in and gets a bearer JWT.

```
POST /auth/sign-up    { firstName, lastName, email, countryCode, password }
POST /auth/sign-in    { email, password } (+ ?platform=app|admin)
POST /auth/refresh    - needs the refresh token, not the access token
GET  /auth/user       - the signed-in user's profile
GET  /auth/access     - every tenant/access this user has, and which is current
POST /auth/access/switch { accessId } - switch which tenant you're acting as
```

Send it as `Authorization: Bearer <token>`. A signed-in user can act on behalf of any app their tenant owns by naming it with the `x-app-id` header (`AppScopeGuard` resolves it); omit the header to fall back to the tenant's default app. Email verification, Google OAuth (`GET /auth/google/sign-in` → redirect), forgot-password, and MFA (`/auth/mfa/*`) all live under `/auth` too — see Swagger for the full flow, it's mostly standard.

Tenant onboarding is a separate step after sign-up — see [§4](#).

API key auth (server-to-server integrations)

An app-scoped secret, `Authorization: Bearer sk_<domain>--<random>`. Issued per app per domain — `POST /tenants/apps/:appId/keys` (session auth only, to create the key in the first place). The key's **app**

and domain are fixed at issue; the `x-app-id` header is ignored when authenticating with a key — you cannot use one key to act on a different app.

Every API-key route also declares a **scope** it requires (`@ApiKeyScopes(...)`), enforced by `ApiKeyScopeGuard`). A key only works on routes whose required scope is in the key's granted list:

Scope	Grants
customers:read / customers:write	Customer CRUD, documents read, account purpose, flags
documents:write	Uploading/updating documents
verifications:read / verifications:write	Starting and reading verification runs, the dashboard endpoints
webhooks:read / webhooks:write	Reading deliveries, requeueing a redelivery
transactions:read / transactions:write	Transaction ingestion and reads

A key with none of the scopes a route needs gets a 403, not a 401 — the key itself is valid, it just isn't allowed to do that.

MFA

GET `/auth/mfa/status` , POST `/auth/mfa/setup` (returns a QR/secret), PUT `/auth/mfa/enable` / PUT `/auth/mfa/disable` (both take a TOTP `token`), POST `/auth/mfa/validate` (completes a sign-in that required a second factor — takes `userId` + `token` , returns the same session shape as sign-in).

3. Public option sets

```
GET /public/misc/options           - every set at once
GET /public/misc/options?keys=a,b - just the named sets
GET /public/misc/options/keys      - just the list of available keys
GET /public/misc/options/:key      - one set
```

No auth required. This is the enum vocabulary the API validates against, served as data specifically so a dropdown never hardcodes a value list that can silently drift from what the server accepts — read it once per page load and cache it (the test console does exactly this in `lib/options.ts`). Each set is `{ value, label }` pairs.

Keys available today: `countries` , `profile-types` , `domains` , `genders` , `business-types` , `business-industries` , `employee-counts` , `shareholder-types` , `individual-verification-types` , `business-verification-types` , `shared-verification-types` , `kyc-document-categories` , `kyb-document-categories` , `individual-document-types` , `business-document-types` , `workflow-statuses` , `workflow-priorities` , `verification-run-statuses` , `verification-outcomes` , `verification-provider-keys` , `verification-reason-severities` , `document-statuses` , `customer-lifecycle-statuses` , `risk-bands` , `risk-decisions` , `risk-priorities` , `risk-factor-types` , `pep-tiers` , `role-permissions` , `team-invite-statuses` .

The response also carries a `constants` object alongside `sets` — currently just `{ riskScore: { min, max } }` , the 0–100 internal scale a `riskScore` value is drawn from (see `risk-model.md` for the externally-facing 0–5 banding).

4. Tenant onboarding

POST /tenants/onboarding (session auth)

Creates the tenant for the signed-in user and returns a fresh access token scoped to it — call this once right after sign-up, before anything else. Body: `name`, `registrationNumber`, `countryCode`, `timezone` required; `yearFounded`, `website`, `tin`, `industry`, `companySize`, `address` optional. Onboarding also provisions the tenant's default app, both sandbox and production API keys (backfilled with every scope), default roles, and default risk/PEP/compliance config — nothing else needs manual setup before a first verification can run.

5. Apps, API keys & configuration

GET/POST/PUT/DELETE /tenants/apps — everything here is **session auth only** (`PermissionsGuard`, requires `tenant-settings:update`), never an API key. This is the dashboard's settings surface for managing the things API keys eventually authenticate as.

- **Apps** — GET /tenants/apps, GET/PUT/DELETE /tenants/apps/:appId, POST /tenants/apps ({ name, description? }). Deleting an app disables it; its customers and verdicts are kept, not deleted.
- **API keys**, per app per domain — GET /tenants/apps/:appId/keys (list, secrets never returned after creation), POST /tenants/apps/:appId/keys ({ name, scopes: TenantApiKeyScopeEnum[] }), returns the secret **once** — copy it immediately), POST /tenants/apps/:appId/keys/:keyId/rotate (new secret, same scopes, old secret dead immediately), DELETE /tenants/apps/:appId/keys/:keyId .
- **Per-app config**, each GET / PUT under /tenants/apps/:appId/... : `risk-factor-configurations` (+ /:slug to update one factor's weight), `risk-score-threshold` (`warningThreshold`, `blockThreshold`), `pep-tier-configurations` (+ /:tier), `notification-preferences` (webhook URL, enabled flag, secret rotation lives at .../webhook/rotate-secret), `compliance-rules` (`kycExpiryDays`, `kybExpiryDays`, `kycDocuments[]`, `kybDocuments[]`, `flaggedCountryCodes[]` — this is what gets snapshotted onto every run, see `risk-model.md`).
- **Webhook deliveries** for the app — GET /tenants/apps/:appId/webhook/deliveries, POST .../deliveries/:id/redeliver — mirrors [§11](#)'s API-key-reachable version but from the dashboard side.
- **App-scope backfill** (operational, rarely needed from a frontend): GET /tenants/apps/scope/status, POST /tenants/apps/scope/backfill .

Team & roles, also session-only, under /tenants/settings : GET/POST /tenants/settings/teams (invite a teammate — { fullName, email, roleId }), POST .../teams/resend, DELETE .../teams/:inviteId/revoke ; GET/POST /tenants/settings/roles-permissions, GET/PUT/DELETE .../roles-permissions/:roleId (note the route's own typo — `permisssons`, not `permissions`), GET .../roles-permissions/options for the permission vocabulary. A role needs `riskLevelMinimum <= riskLevelMaximum` or the write 400s.

Business information: GET/PUT /tenants/settings/business-information .

6. Domain (sandbox / production)

Every tenant operates in exactly one of two domains at a time, and it's first-class throughout the API — customers, workflows and keys are all domain-scoped, and a sandbox customer is invisible to a production request and vice versa.

- An **API key** is bound to one domain at issue (`sk_sandbox-...` / `sk_production-...`) — no header needed, no way to switch.
- A **session** user switches with `POST /tenants/settings/domain/switch ( { domain: "sandbox" | "production" } )`, which re-issues the access token with the new domain baked in.

Sandbox resolves every provider call to deterministic simulators — no credentials needed, nothing billed, nothing sent to a real watchlist. Build and demo against it freely. The verdict keys off the last four digits of whatever identifier you send, and the AML simulator flags a name containing "sanction" or "pep".

7. Customers — KYC & KYB

Two parallel resource trees, `/customers/kyc` (individuals) and `/customers/kyb` (businesses) — same shape throughout, business fields substituted for individual ones.

List & stats

```
GET /customers/kyc          scope: customers:read
GET /customers/kyc/stats    scope: customers:read
GET /customers/kyb          (same shape)
GET /customers/kyb/stats
```

List query params: `page`, `limit`, `status` (`CustomerLifecycleStatusEnum` — omit it to exclude offboarded customers by default), `search` (name/ business-name, email, phone), `dateFrom` / `dateTo` (ISO strings, filters `createdAt`).

As of 2026-09-14 the list row no longer carries `priority`, `assignedTo`, or `timeInQueueMs`, and there's no `priority` filter here — those are workflow-queue fields now served by `/verifications/kyc|kyb`, see [§8](#) and `verification-list-endpoints.md`. If a screen needs both a customer's own fields and its queue state in one row, join client-side on `customerId`.

Stats: `{ successful, pending, highRisk, rejected }` — counts derived from each customer's most recent workflow, not the customer row itself, so a customer who never started a verification still counts toward `pending`.

CRUD

```
GET    /customers/kyc/:customerId
POST   /customers/kyc          scope: customers:write
PUT    /customers/kyc/:customerId
DELETE /customers/kyc/:customerId { reason? } — offboards, does not hard-delete
```

Create body (individual): `firstName`, `lastName`, `dob` (ISO date), `gender`, `email`, `phone`, `countryCode`, `address` — required fields on the compliance-domain contract; the address object's shape is `{ houseNo?, street?, city?, state?, stateCode?, country?, countryCode?, zipCode?, formatted? }`, all optional individually. Business create body: `businessName`, `registrationDate`, `contactEmail`, `contactPhone`, `countryCode`, `industry?`, `address`. `tenantId` / `domain` / `appId` are never sent by the caller — they're resolved server-side from auth.

Delete is a soft lifecycle transition (`status` → `offboarded`), not a row deletion — an offboarded entity is never silently re-onboarded by a later clean run.

Documents

```
GET /customers/kyc/:customerId/documents
POST /customers/kyc/:customerId/documents      scope: documents:write
PUT /customers/kyc/:customerId/documents/:documentId
GET /customers/kyc/:customerId/documents/requirements
```

Documents are sent **base64-encoded inline in JSON**, not multipart — the upload is a data URI. Client-side limits (also enforced server-side, so check before sending): **2MB**, and only `application/pdf`, `image/jpeg`, `image/png`, `image/webp`, `image/heic`, `image/tiff`. `.../documents/requirements` returns the tenant's configured required categories for this customer with which are already satisfied — render this before letting someone start a verification, so a missing document surfaces before the run rather than as a failed task after.

Account purpose

```
GET /customers/kyc/:customerId/account-purpose
PUT /customers/kyc/:customerId/account-purpose
```

What the customer declared they'd use the account for — feeds C2's transaction-monitoring anomaly detection (real activity compared against this declaration). Not required to start a verification.

Flags

```
PATCH /customers/kyc/:customerId/flags/:flagId      scope: customers:write
      { status: "warning" | "investigating" | "revised", note? }
```

Flags move forward through `warning` → `investigating` → `revised` only — a skipped or backwards transition 400s as `bad-transition`. `note` is required when moving to `revised`.

KYB-only: shareholders

```
GET /customers/kyb/:customerId/shareholders
POST /customers/kyb/:customerId/shareholders
PUT /customers/kyb/:customerId/shareholders/:shareholderId
GET /customers/kyb/:customerId/shareholders/:shareholderId/documents
POST /customers/kyb/:customerId/shareholders/:shareholderId/documents
```

A KYB run screens every shareholder alongside the business itself (sanctions

- PEP only — shareholders don't get identity/document checks, those apply to the company). Have today: name, role, share percentage/count, type, date appointed, email, phone, country, address.
Missing: nationality, ID type/number, share class — `kyb-review.md` has the full gap list.

8. Verifications

```

POST /verifications/kyc      scope: verifications:write  { customerId,
verificationTypes: [{ type, providerKey? }] }
POST /verifications/kyb      (same shape, business types)
GET  /verifications          scope: verifications:read
GET  /verifications/kyc      - added 2026-09-14, see below
GET  /verifications/kyb      - added 2026-09-14, see below
GET  /verifications/:workflowId
GET  /verifications/available-checks  - see §9

```

Starting a run

`verificationTypes` is an array — each entry `{ type, providerKey? }`. Omit `providerKey` to let the platform pick the provider(s) automatically (as of 2026-09-14, **every provider the catalogue lists for that type and country runs**, not just one — see below); pin one to force a single specific provider. The server also silently adds any mandatory checks (sanctions + PEP screening today) the request omitted — the response's `mandatoryAdded` array says what got added. Requesting a type your customer's country can't answer 400s before anything starts (the UK has no national ID card, for example) — check `available-checks` first if the UI lets someone pick freely.

Response: `{ workflowId, workflow, run, mandatoryAdded }` — `workflow` is the same shape returned by the list endpoints below, `run` is `{ run, tasks }` from compliance-domain, one task per requested check (plus one per shareholder-screening pair on a KYB run).

Multi-provider verification (2026-09-14)

A check now runs against **every eligible provider**, not one, and the task's final outcome is a reconciliation across every provider that answered — a provider that couldn't be reached is excluded from the vote entirely (it isn't a verdict), any real `failed` outranks everything, then `review-required`, and only a unanimous pass across every provider that did answer settles `passed`. **Nothing about the request or response contract changes because of this** — `outcome` is still one of `passed | failed | review-required | pending | skipped | error`. The one thing worth knowing: a task's `reasons[]` can now contain reasons from more than one provider, concatenated, with no per-reason attribution in the payload today. Full detail in `verification-list-endpoints.md`.

Reading a run

`GET /verifications/:workflowId` → `{ workflow, run: { run, tasks } | null, risk, events }`. `run` is null until compliance-domain has picked the workflow up (should be near-instant); poll or re-fetch until `workflow.status` reaches a terminal value (`verified / failed`) — there's no push notification for this today, the test console polls every 3s. Every task carries `outcome`, `reasons[]` (always populated, even on a pass — every verdict has to explain itself), `providerKey`, `providerReference`, `attempts`. `risk` is null until risk-decisioning has scored the run; `events` is the full audit trail (`action`, `actor`, `fromStatus`, `toStatus`, `detail`, `createdAt`) — this is what a compliance officer reads to answer "why is this in review."

`workflow.verificationTypes` is an **array** (`string[]`) — a run started with three checks produces one workflow row naming all three, not three rows. Document Type as shown in a list ("National ID + Selfie") is a frontend-side join of this array against the human labels from `/public/misc/options?keys=individual-verification-types` (or the richer per-check labels `available-checks` returns) — the server doesn't compute a composite label on the generic endpoints, only on the two below.

GET /verifications/kyc and GET /verifications/kyb

The workflow-primary queue view — one row per verification, not per customer, enriched with the customer's name/country in one batched lookup and a friendly `documentType` label. Filters: `page` , `limit` , `status` , `priority` , `verificationType` (matches a workflow whose `verificationTypes` includes it), `dateFrom` / `dateTo` , `search` (id/displayId/ providerSubjectId only — **not** customer name). Full response shape, field-by-field, is in `verification-list-endpoints.md` ; the short version: everything a generic workflow row has, plus `displayId` , `customerName` , `country` , `documentType` .

`GET /customers/kyc|kyb` is the roster (who); this is the queue (what's happening to them). A screen showing both priority/assignment/time-in-queue *and* full customer detail in one place has to combine the two.

9. Available checks

```
GET /verifications/available-checks?countryCode=NG&profileType=individual
```

No auth required, no tenant scoping. What the applet should render as selectable checks for a given country/profile — each entry names the check, whether it's offerable there, its friendly label and authority ("National Identification Number" , "National Identity Management Commission"), and which providers the catalogue lists for it (`eligibleProviders`). Call this before rendering a check-selection UI rather than hardcoding a list — a check absent for a country genuinely isn't offerable there (the UK has no national ID), and requesting it anyway 400s at verification-start time.

10. Dashboard

All under `/dashboard` , scope `verifications:read` , both auth modes work (`CombinedAuthGuard`).

```
GET /dashboard/summary          - { total, successRate, failureRate,
pendingReviewRate } shape; verified vs failed vs pending workflow counts
GET /dashboard/high-risk-alerts - paginated, workflows needing urgent attention
GET /dashboard/verification-types - counts grouped by check type (a workflow with 3
checks contributes to all 3 counts, not 1)
GET /dashboard/activity          - paginated, most-recent-first activity feed across
the tenant
GET /dashboard/api-calls?year=2026 - successful/failed request counts by month for the
given year
```

`verification-types` returns `[{ type, label, count, percentage }]` , sorted by count descending. `activity` items describe what happened (`verification.started` , `risk.decision` , `aml.alert.flagged` , `transaction.monitoring.alert` , ...) in the same vocabulary as a workflow's own event trail — see `describeActivity()` 's cases if you need the exhaustive list, or just render `detail` generically.

11. Webhooks

Two ways to reach delivery history — `GET /webhooks/deliveries` (API-key reachable, scope `webhooks:read` / `webhooks:write` , always scoped to the key's own app) and the dashboard mirror under `/tenants/apps/:appId/webhook/*` (session auth, §5). Same underlying data either way.

```
GET /webhooks/deliveries ?
workflowId=&status=pending|delivered|failed|dead
GET /webhooks/deliveries/:id
POST /webhooks/deliveries/:id/redeliver scope: webhooks:write
```

A delivery record carries the payload, response, and last error — enough to debug a missed callback without dashboard access, which is the whole reason this exists as its own API-key-reachable surface. Redelivering re-sends the exact original payload; the receiver sees it as a fresh delivery even though nothing changed on our side.

What gets sent, and how to verify it

Two events today: `workflow.verified` , `workflow.failed` — fired once, when a workflow reaches that terminal status (never for `in-review` ; that's the flowchart's EDD hold and must not collapse into either verdict). Configure the URL and enable/disable delivery under `/tenants/apps/:appId/notification-preferences` .

Every delivery carries `x-unifycomply-signature: t=<unix-seconds>,v1=<hex-hmac-sha256>` , computed over `${timestamp}.${rawBody}` with the app's webhook secret (get/ rotate it at `.../webhook/rotate-secret` — rotating invalidates every signature computed with the old one, so redeliveries in flight during a rotation will fail verification with the new secret). This is the Stripe convention: verify by recomputing the same HMAC over the exact raw request body (don't re-serialize a parsed JSON object — whitespace differences break the signature) and comparing in constant time, and reject anything with a timestamp outside a tolerance window you choose to defend against replay.

Payload shape: `{ event, workflowId, timestamp, data: { workflow, customer, run, tasks, risk } }` — `workflow.verificationTypes` is the array form (see [§8](#)), `tasks` is a slimmed per-task list (`verificationType`, `outcome`, `reasons`, `providerKey`, `providerReference`), and `risk` carries the full decision (`decision`, `score`, `band`, `priority`, `warningThreshold`, `blockThreshold`, `canApprove`, `requiresEscalation`, `contributions`).

12. Transactions

Transaction-monitoring's whole client-facing surface, proxied by core-platform — nothing is stored here, every call fans out over NATS.

```
POST /transactions scope: transactions:write
GET /transactions scope: transactions:read ?
customerId=&dateFrom=&dateTo=&page=&limit=
GET /transactions/:id scope: transactions:read
```

Create body: `customerId` , `profileType` , `direction` , `amount` , `currency` , `channel` , `reference` , `occurredAt` (ISO string) required; `corridorCountryCode` , `counterparty` , `metadata` optional. **reference must be unique per caller and is the idempotency key** — retrying a create with the same `reference` returns the original row instead of creating a duplicate. This matters for a near-real-time feed that retries on timeout: without a stable caller-supplied reference, a retry becomes a duplicate transaction rather than a safe no-op, so always generate `reference` once per real transaction, not once per HTTP attempt.

Feeding transactions in is what powers C2's anomaly detection — real activity compared against the customer's declared account purpose ([§7](#)) on a trailing 30-day window. A breach surfaces as a

`transaction.monitoring.alert` activity-feed entry and flips an already-verified customer's workflow to `in-review` ; there's no direct "get anomalies" endpoint today, it's activity-feed and dashboard-only.

13. User profile

```
GET /users/me/profiles
PUT /users/me/profiles
```

Session auth only. The signed-in user's own profile within their current access (tenant + role) — distinct from `GET /auth/user` , which returns the bare user record without access/tenant context.

14. Settings — deprecated tenant-wide routes

`/tenants/settings/*` has a long tail of routes (`api-key` , `risk-factor-configurations` , `risk-score-threshold` , `pep-tier-configurations` , `notification-preferences` , `webhook/rotate-secret` , `webhook/deliveries` , `compliance-rules` , and their sub-routes) all marked `deprecated: true` in Swagger — each has a named successor under `/tenants/apps/:appId/...` (§5) now that configuration is per-app rather than tenant-wide. They still work (reading/writing the tenant's *default* app's config), but **don't build against them** — use the app-scoped route named in each deprecation notice.

`GET/POST /tenants/settings/teams` , the roles-permissions routes, and `business-information` are **not** deprecated — those stay tenant-wide by design, teams and roles aren't an app concept.

`domain/switch` also lives under `/tenants/settings` — see [§6](#) — and is current, not deprecated.